

OnlyFriends

Integritetsbyrå

Abstract

OnlyFriends is a social media platform for staying connected with your real-life friends. It aims to be privacy friendly through e2e encryption and minimization of metadata leaks to third parties.

Contents

1. Terminology	1
2. Device Chains	1
2.1. Types	1
2.2. Adding Siblings	2
2.3. Synchronizing	3
2.4. Removing Siblings	4
2.5. Adding Friends	4
2.6. Removing Friends	4
2.7. Forks and Time Travel	5
2.8. State Resolution	5
3. Networking	5
References	5

1. Terminology

- **App** An implementation of OnlyFriends.
- **User** An individual, described by a set of *device chains*.
- **Device** One app instance, belonging to one specific *user*.
- **Sibling** A *device* belonging to the same *user*.
- **Device Chain** A linked list of *Entrys*, each pointing to an optional *Body*, expressing the state of a single *device*.
- **Entry** A cryptographically signed data type that is the building block of the *device graph*.

2. Device Chains

A device is described only by a device chain. The device chain is an append-only linked list, where each entry is cryptographically signed using ed25519ph [1]. Everything a user does is included in the device chain: Every post, every comment, every friend.

2.1. Types

Entry is the root type of the device chain. Postcard is used for serialization.

```
type EntryId = Sha512<Entry>;
```

```
struct Entry {  
    /// Reference to the previous [`Entry`]  
    prev: EntryId,  
    /// Kind, ID, and length of the [`Body`] of this entry, if any.  
    body: Option<(BodyContentKind, BodyId, u64)>,  
    /// Tails of sibling device chains. Used as a vector clock.  
    // TODO: This makes the `Entry` variable-sized. Having an upper-bound on size would  
    be nice for synchronization guarantees.  
    sibling_tails: Map<DeviceId, EntryId>,  
}
```

```
type BodyId = Sha512<Body>;
```

```
struct Body {
    content: BodyContent,
}
```

```
enum BodyContent {
    Root { /* ... */ },
    IntroduceSibling { /* ... */ },
    Tombstone { /* ... */ },
    Befriend { /* ... */ },
    Unfriend { /* ... */ },
    Post { /* ... */ },
    Reaction { /* ... */ },
    // etc
}
```

```
/// The Device ID is the Entry ID of the device chain root node.
```

```
type DeviceId = EntryId;
```

The ID types are SHA512 hashes of the identified value. Each Entry references its own Body by ID, as well as the previous Entry. This creates a linked list of entries – the device chain, which facilitates a common view from any given device. More details in Section 2.3.

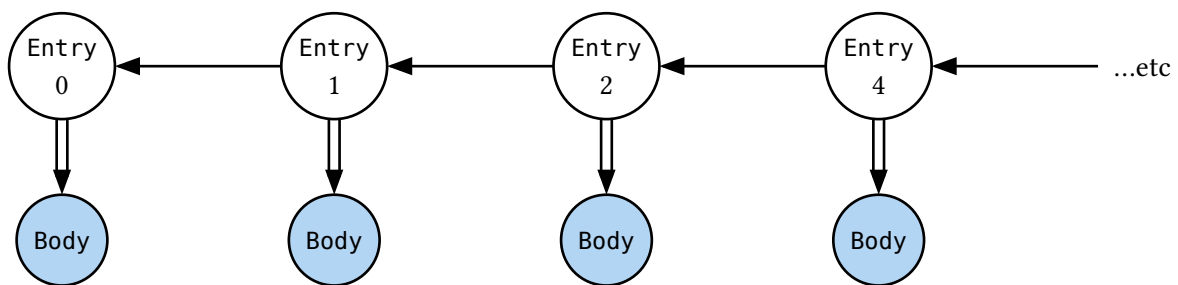


Figure 1: A Device Chain

Bodys and Entrys serve different purposes:

- An Entry is small and signed. It's used to quickly synchronize the shape of a device chain. They also indicate Body *kind* and *length*.
- A Body can contain rich data, like user text and media. Data that may be large in size, which a device may wish to discard or ignore to save on storage or network usage.

2.2. Adding Siblings

A user who has a device A_1 may introduce a second device A_2 by having it initialize its own device chain, introducing A_1 and having A_1 introduce it. A_1 and A_2 are siblings. Sibling-introductions are a special Body which includes the siblings DeviceId (*TODO* and public keys?).

In Figure 2, 2_{A_1} is an Entry from A_1 introducing A_2 as a sibling. This Entry is propagated to Alice's friends, allowing them to recognize A_2 . Likewise, having 0_{A_2} vouch for A_1 's device chain, allows future friends introduced by A_2 to recognize the siblings.

TODO Should a hypothetical sibling A_3 , also introduced by A_1 , strive to vouch for A_2 in its device tree to make sync easier? Perhaps a device must vouch for its sibling before including it in its vector clock?

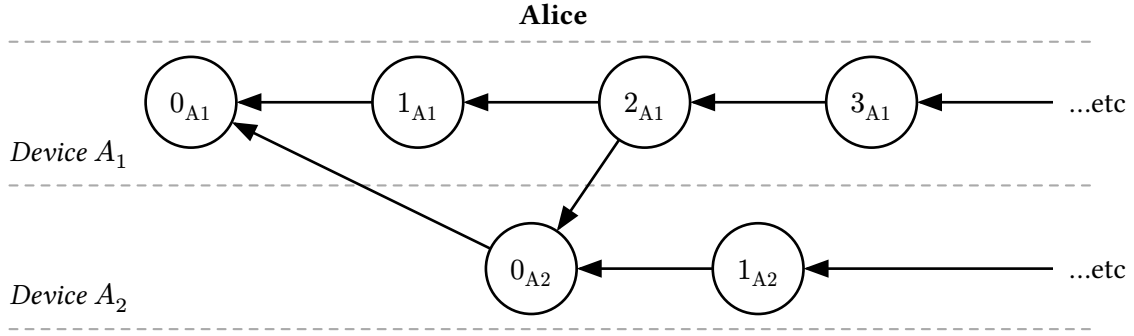


Figure 2: Introducing a Sibling

2.3. Synchronizing

Let Alice and Bob be users. When Alice wants to synchronize with Bob, she must first establish a line of communication, this is described in Section 3. Figure 3 gives an example of a subsequent conversation between Alice's device A_1 , and Bob's device B_1 . In this example, Alice has a second device A_2 .

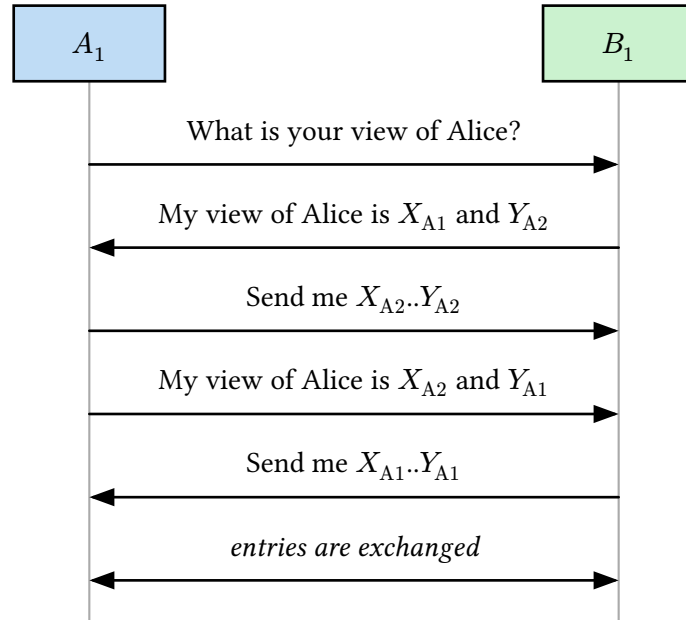


Figure 3: Synchronizing Entries with a friend

In Figure 3, B_1 has seen a more recent Entry from A_2 than A_1 has. We allow a device (A_1) to synchronize Entries from any sibling (A_2) via any friend (Bob). Entries are small and cheap to send, and they are sent from oldest (X_{A1}) to newest (Y_{A1}), such that the signatures of each Entry can be verified immediately on reception.

After both devices are up to speed on Entries, They *may* ask each other to send Bodys as well.

Note that devices are not required to comply with synchronization requests. Apps may implement per-friend transfer quotas, to minimize the usefulness of exploiting friends for cheap storage.

If a device want's to append Entries to its own device chain, it should generally defer from doing so until it has synchronized with at least one other device (friend or sibling). This can look like Figure 3, where Alice's device starts by asking Bob's device about *its* view of Alice's devices. This, way, the device has a more up-to-date view of its siblings when the Entry is added, which can help reduce state conflicts as described in Section 2.8.

2.4. Removing Siblings

Removing a device is mostly a formality. There is no harm in simply “forgetting” a device by locally deleting its private keys, but formally removing it allows other devices to present it as removed, improving UX.

A device can be removed in two ways, but both ways require a sibling.

The first way, is for the device to append a tombstone Entry to its device chain. Other devices must reject Entries that append to a tombstone Entry, so this effectively finalizes the device chain.

A device can remove itself iff it has an active connection to a sibling. Without a sibling, removal would likely be futile since a user might promptly delete the device app, leaving little opportunity for the tombstone Entry to be propagated to siblings (or friends) in the normal fashion.

A self-removal is performed like so:

- The doomed device establishes a direct connection to a sibling device.
- The doomed device appends a tombstone message to its own device chain.
- The doomed device sends it to the sibling device, which is responsible for propagating the tombstone message to friends and other siblings.

The other way to remove a device, is by its sibling issuing a sibling-revocation-Entry. This can be useful when a device is lost. The sibling-revocation-Entry references the siblings DeviceId, and its latest known Entry.

2.5. Adding Friends

A friend is added by a befriend-Entry. The Body is encrypted, and includes the device chain root (0th Entry and Body) of the friend.

When friends are added, two devices from different users exchange their device chain roots (and the root of the original device) over a secure side channel. The 0th Body contains a device’s public keys, which facilitates secure communication over normal channels.

After the 0th is exchanged, synchronization happens as detailed in Section 2.3.

TODO With befriends/unfriends being present in the device chain, this means that your friends can know roughly how many friends you have. We can avoid this metadata leakage by maintaining a parallel encrypted device chain that we don’t share with friends, but sync would be more fragile, since we couldn’t bounce the device chain via friends. Presently, we leak this metadata:

- How many friends we have
- When any friend was added

2.6. Removing Friends

A friend is removed by an unfriend-Entry. Its Body is encrypted, and references the Entry which added the friend.

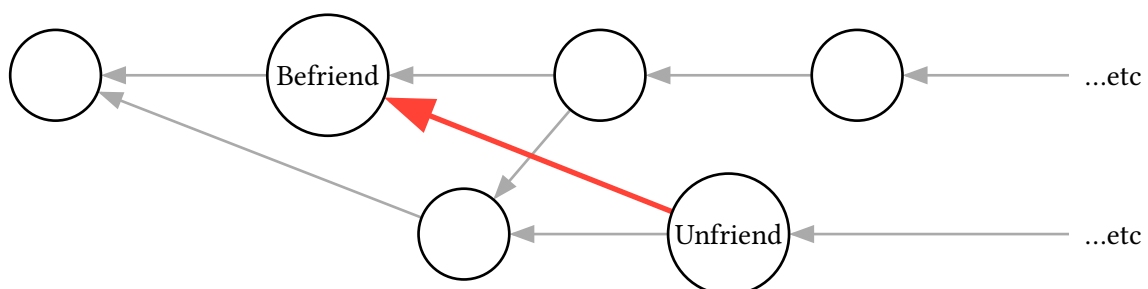


Figure 4: Befriend and Unfriend

TODO: Improve Figure 4

When a friend is removed, devices must cease all communication with friend devices.

TODO Inform friend that it has been removed? If we don't do this, a friend might try to connect with you forever in futility.

2.7. Forks and Time Travel

It is possible for malicious or malfunctioning devices to go back and create different, diverging, device chains (a fork). Protecting against this is not a priority, since there is little practical harm in a device attempting to rewrite its history, or trying to present different views to different friends.

Still, for the purposes of local consistency, a device must reject any incoming Entries that do not strictly append to a device chain. This way, forks are prioritized by their time of receipt, with the premise being this: Once a user has seen something (e.g. a post, a comment), it must not change.

Additionally, if a device observes a forked device chain for a friend, the app *may* choose to automatically generate a unfriend Entry, since the friend's device is either broken or malicious.

2.8. State Resolution

Two siblings may independently change the same user state, creating a conflict. For example, both devices may update the user's profile picture. These conflicts are resolved by a vector clock[2], and if the clock is inconclusive, by the lowest DeviceId.

When a device receives a new sibling Entry from a sibling, it must, if possible, include the tail EntryId of the sibling device chain in the next Entry it appends to its own device chain. By referencing earlier Entries from siblings in this fashion, we establish a partial ordering of Entries. This is possible because the set of all device chains of a user make up a directed acyclic graph.

Furthermore, the vector clock is transitive: Device chain A_1 must not reference the latest entry of its sibling A_2 , without also referencing the latest entry of *every additional sibling* referenced by A_2 .

3. Networking

TODO

References

- [1] IRTF, "RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)." [Online]. Available: <https://www.rfc-editor.org/info/rfc8032/>
- [2] Wikipedia, "Vector clock." [Online]. Available: https://en.wikipedia.org/wiki/Vector_clock